PATHFINDER

Opportunity discovery and career readiness, made clearer.

PROJECT DOCUMENTATION

Project Requirements & Technical Reference

Version 2.0 | November 2026

Product definition, requirements, architecture, data model, AI guardrails and MVP delivery reference.

Document purpose and source-of-truth rules

This document defines PathFinder's product scope and technical direction. It is intended to guide the tutor-template-based project brief, Kiro's specification phase, architecture review and implementation.

Product scope is not open to silent reinterpretation. Kiro must preserve the approved product behaviour and requirement identifiers. If something is missing, contradictory or technically infeasible, it must identify the issue and ask for a decision rather than inventing a resolution.

Distinguish decisions from recommendations. Confirmed decisions are binding. Recommended technologies and unresolved implementation details must be identified as such until approved.

Confirmed decisions

Area	Decision
Architecture	Modular monolith: one application/codebase and initial deployment, internally separated into feature modules.
Background processing	No separate worker application in the MVP. Introduce one only when measured workload or operational needs justify it.
LLM provider	DeepSeek is the sole LLM provider in the MVP, accessed through the Vercel AI SDK and `@ai-sdk/deepseek`. No multi-provider routing.
Eligibility	Deterministic application logic evaluates structured mandatory criteria. The LLM cannot decide or override eligibility.
Opportunity data	Admin-curated and verified opportunities for the MVP; no broad automated crawler at launch.
Application submission	Users apply through the official provider channel. PathFinder does not submit applications.
Audience and geography	Nigerian undergraduates and recent graduates; global opportunities where applicant eligibility supports the user's profile.

Recommended technical baseline, pending final implementation review

The prior reference recommends Next.js App Router, TypeScript strict mode, PostgreSQL, Drizzle ORM, Tailwind CSS, shadcn/ui and Zod. These are a recommended baseline, not all individually confirmed choices. Exact versions, authentication provider, database hosting, package manager and deployment configuration must be confirmed before implementation.

1. Product overview

PathFinder is an AI-assisted opportunity discovery and career-readiness platform for Nigerian university undergraduates and recent graduates. It helps users discover scholarships, SIWES/IT placements, professional and research internships, fellowships, graduate programmes, funded training and leadership opportunities around the world where applicants with their nationality and profile may be eligible.

Visitors can browse without an account. Registered users can build an academic and professional profile, receive recommendations, assess stated eligibility, understand missing information or qualifications, create preparation plans, save opportunities and track application progress. Applications are completed through the official provider website or portal.

PathFinder is an opportunity intelligence and recommendation system powered by AI, not primarily a general-purpose chatbot.

1.1 Target users and scope

- Undergraduates: scholarships, SIWES/industrial training, internships, research placements, skills programmes and development opportunities.
- Recent graduates: fellowships, graduate programmes, internships, funded training and further-study opportunities. NYSC status may affect eligibility and should be captured where relevant.
- Opportunity destinations are global, but the platform must not infer that a programme accepts Nigerians merely because it is international.
- Applicant nationality, applicant residence and opportunity destination are separate data dimensions.

• Funding can include fully funded, partially funded, stipend-supported, paid, unpaid, tuition-free, travel-supported, accommodation-supported and self-funded opportunities.

1.2 Product value

Value	What PathFinder provides
Discovery	Searchable opportunities across categories and destinations.
Clarity	Structured eligibility information, including unknown and review-needed facts.
Relevance	Recommendations based on academic, skill and career profile.
Preparation	Actionable readiness guidance without confusing preparation gaps with formal ineligibility.
Trust	Official provider links, source provenance and verification timestamps.

2. Main user journey

Step	Behaviour
1. Discover	Visitor browses, searches and filters public opportunities.
2. Inspect	Visitor reviews provider, eligibility, funding, deadline, source and application link.
3. Register	User creates an account and completes an academic/career profile.
4. Assess	System evaluates structured eligibility criteria and calculates relevance separately.
5. Understand	User sees met, unmet, unknown and review-needed criteria.
6. Prepare	On request, AI explains computed results and creates a grounded readiness plan.
7. Track	User saves the opportunity and records application progress.
8. Apply	User follows the official provider link; PathFinder does not submit the application.

Public browsing does not require registration. Personalised assessment, saved opportunities and application tracking require an account. Profile onboarding should collect only information that materially supports eligibility, relevance or preparation: education status, institution, course/field, academic level, graduation year, nationality, residence, NYSC status where applicable, skills, experience and career goals.

Users may record progress using an agreed status model. PathFinder must not claim that an application was submitted unless the user records that status.

3. Eligibility, relevance and readiness

Operation	Question	Mechanism
Eligibility	Does the user meet stated mandatory conditions?	Deterministic rules for structured criteria; explicit unknown/review states otherwise.
Relevance	How closely does the opportunity align with the user's profile and goals?	Transparent ranking using approved profile and opportunity attributes.
Readiness	What can the user improve to prepare a stronger application?	AI-assisted guidance grounded in verified requirements and known profile gaps.

3.1 Eligibility outcomes

Outcome	Meaning
Eligible	All known mandatory conditions are satisfied and no required condition remains unresolved.
Likely eligible	Known conditions appear satisfied, but important details remain unconfirmed.
Needs review	Criteria or profile data are ambiguous or insufficient for a dependable result.
Not eligible	At least one known mandatory condition is not met.

Missing information is unknown, not a pass. A known mandatory failure must not be hidden by a high relevance score. A relevance score is not an eligibility probability and does not predict selection, admission or funding.

3.2 Criterion evaluation

• Each criterion should have a type, operator, required value, mandatory/preferred status and source reference where possible.

• Return a criterion-level result such as met, not met, unknown or needs review, with a concise reason.

• A known mandatory failure produces Not eligible. Unknown mandatory information prevents a definitive Eligible result.

• Natural-language criteria that cannot be safely structured must remain flagged for review; do not invent a machine interpretation.

3.3 Relevance and readiness

Relevance may consider education, skills, experience, interests, career goals and approved profile attributes. Ranking factors must be understandable; do not adopt arbitrary weights without rationale, tests and approval. Readiness plans may cover skills, portfolio, documents, experience and preparation steps. Each task must be grounded in verified requirements or labelled as general advice. Users should be able to review, edit and mark tasks complete.

4. AI behaviour and DeepSeek

4.1 Provider configuration

DeepSeek is the sole LLM provider for the MVP. All LLM calls must pass through the Vercel AI SDK using `@ai-sdk/deepseek`. Use the appropriate standard SDK interfaces, such as `streamText` for streamed text and structured-generation interfaces such as `generateObject` where supported by the selected SDK version. Verify package/API compatibility before implementation.

There is no multi-provider routing logic in this phase. Keep provider-specific code behind a small internal AI service boundary so a future provider can be added without scattering DeepSeek-specific calls throughout the application. This is a maintainability boundary, not a multi-provider framework.

4.2 Allowed responsibilities

• Explain already-computed eligibility results.

• Explain relevance factors without presenting them as selection probabilities.

• Generate structured readiness plans from verified opportunity facts and relevant profile data.

• Assist with interpretation of ambiguous natural-language criteria, but leave ambiguous cases flagged for human review.

4.3 Prohibited behaviour

• Never decide, calculate, override or certify formal eligibility.

• Never invent requirements, nationality/residence rules, age limits, deadlines, funding, stipends, links, documents or provider decisions.

• Never turn missing information into a positive claim.

• Never promise admission, employment, scholarship awards, visa approval or selection.

• Never expose one user's private profile, saved opportunities or application data to another user.

4.4 Grounding, validation and failure

Supply only relevant verified opportunity facts, source context, computed eligibility results and the minimum profile attributes needed. Validate structured output at runtime before displaying or saving it. Reject or flag malformed output and unsupported claims. Preserve source references and assessment timestamps. Public browsing and deterministic eligibility must remain usable if DeepSeek is unavailable; AI features must fail clearly rather than fabricate a fallback answer. API keys remain server-side.

5. Opportunity curation and verification

The MVP uses manual admin curation. Administrators capture structured details, check official provider sources, record evidence and verification metadata, then publish only after review. Broad crawling, scraping and external data partnerships are deferred.

Data dimension	Information to preserve
Identity	Title, provider/organisation, category and official programme page.
Eligibility	Nationality, residence, education level, field, age, experience, language, documents and other stated criteria.
Location	Destination, city, remote/in-person/hybrid delivery mode.
Timing	Opening date, deadline, timezone context, programme dates, rolling/no-deadline status.
Funding	Each coverage component, amount, currency, frequency and evidence state.
Internship	SIWES/IT relevance, duration, work arrangement, paid status, academic level and enrolment.
Verification	Source URL, last checked date, reviewer, outcome, evidence notes and lifecycle status.

Unknown-data rules

- Do not infer Nigerian eligibility from an international brand or host.
- Do not infer funding from a programme title.
- Do not infer an open deadline from an old listing.
- A page loading successfully does not prove its substantive details are current.
- Where evidence is absent, conflicting or unclear, preserve the uncertainty.

Lifecycle states

State	Meaning
Draft	Captured but not reviewed or publicly visible.
Under review	Being checked against its source.
Published / open	Approved for public display; source supports current open status.
Upcoming	Future programme/cycle confirmed but applications not yet open.
Closing soon	Open and approaching its stated deadline.
Closed / expired	Application window ended or source confirms closure.
Unknown / needs recheck	Current status cannot be established reliably.
Archived	Removed from active discovery but retained for history/audit.

Verification records should preserve who checked the listing, when, what was checked, the outcome and supporting notes. Prefer official provider sources when facts conflict. Review cadence is an operational policy to define based on curation capacity; do not invent a universal interval.

6. Conceptual relational data model

This is an entity inventory, not a final migration-ready schema. Kiro must propose a normalised MVP schema, define relationships, keys, constraints, indexes and deletion behaviour, and explain any consolidation or deferral before implementation. Do not force every concept into its own table and do not impose an arbitrary five-table limit.

Entity / concept	Purpose and representative data
users	Application identity, email, display name, auth-provider subject and timestamps; credentials managed by auth provider where possible.
profiles	User type, nationality, residence, education status, institution, course, academic level, graduation year, NYSC status and goals.
skills / user_skills	Canonical skills and profile-skill relationship; proficiency/experience where useful.
experiences	Projects, employment, volunteering, research and other experience.

Entity / concept	Purpose and representative data
organizations	Opportunity provider/host organisation, website and country.
opportunities	Title, description, category, lifecycle status, delivery mode, official URL, deadline and verification state.
opportunity_categories	Category taxonomy and optional parent-child hierarchy.
opportunity_locations	Destination and delivery location, distinct from applicant nationality/residence.
applicant_eligibility	Applicant nationality/residence restrictions and notes.
eligibility_criteria	Structured criteria, operator, required value, mandatory/preferred flag and source reference.
funding_components	Tuition, stipend, salary, accommodation, travel, fees; amount, currency, frequency and evidence state.
opportunity_sources	Official/supporting source URLs, source type, checked timestamp and evidence notes.
verification_events	Admin reviewer, verification timestamp, outcome and notes; audit history.
eligibility_assessments	User/opportunity assessment, criteria version, status, criterion results and timestamp.
match_results	User/opportunity relevance score, factors and calculation timestamp; not eligibility probability.
saved_opportunities	User bookmarks and saved timestamps.
applications	User application progress, status, applied date and private notes.
action_plans / tasks	Readiness plan metadata and tasks, priority, status and optional due date.

Schema design review gate

- Present entity/table inventory and rationale.
- Specify primary/foreign keys, uniqueness/check constraints and indexes.
- Explain cardinality, ownership boundaries and admin access.
- Explain unknown/not-stated values, funding components and geographic eligibility dimensions.
- Define deletion, archival and audit-history behaviour.
- Identify entities combined or deferred and why.
- Obtain project-owner approval before migrations are implemented.

7. System architecture and technical baseline

7.1 Modular monolith

One application/codebase and one initial deployment, with internal feature modules and a shared relational database. Domain logic stays out of UI components. Modules communicate through typed interfaces. No microservices are required for the MVP.

Module	Responsibility
Identity	Authentication, session identity and account lifecycle.
Profiles	Profile onboarding, education, skills and experience.
Opportunities	Catalogue, details, categories, funding, locations and search.
Eligibility	Deterministic criteria evaluation and criterion-level outcomes.
Matching	Relevance calculation, ranking factors and explanations.
Readiness	DeepSeek-backed explanations and structured preparation plans.
Applications	Saved opportunities and application progress.
Verification / Admin	Review, publication controls, source records and audit events.
Notifications (later)	Opt-in reminders if added in a later scope.

7.2 Request and AI flow

Browser → Next.js application → feature module(s) → PostgreSQL. AI requests additionally call the internal AI service → Vercel AI SDK → DeepSeek. Opportunity facts enter through admin curation and verification. This is a logical flow, not a commitment to separate servers.

7.3 Background processing

No separate worker application or queue is required in the MVP. Keep initial processing in the main application and use platform-appropriate scheduled mechanisms only when an actual scheduled task is introduced. Add a worker, Redis or BullMQ only when measured workload, retry requirements or orchestration needs justify the operational cost.

7.4 Recommended stack and unresolved implementation choices

Layer	Direction	Status
Framework	Next.js App Router	Recommended baseline; exact version to confirm.
Language	TypeScript strict mode	Project language.
UI	Tailwind CSS + shadcn/ui	Recommended.
Database	PostgreSQL	Primary relational store.
ORM	Drizzle ORM	Recommended typed schema/migrations.
Validation	Zod	Recommended runtime validation.
AI	Vercel AI SDK + DeepSeek	Provider decision confirmed; verify SDK version/API.
Authentication	Choose one provider	Auth.js and Supabase Auth previously considered; unresolved.
Hosting	Vercel + managed PostgreSQL candidate	Not confirmed.
Package manager	Choose one and commit lockfile	Not confirmed.

7.5 Excluded unless justified

- Microservices and multi-provider LLM routing.
- Separate worker, Redis and BullMQ.
- Vector database, pgvector, advanced RAG and vector retrieval unless a demonstrated requirement emerges.
- CV/document upload and parsing pipeline or S3 ingestion architecture.
- Large-scale crawling, automated application submission, WhatsApp reminders, CV/cover-letter generation, AI interview coaching, employer dashboards, university integrations and multi-agent orchestration.

8. Project folder structure

The tree uses the clear nested-tree style of the Fingest reference, adapted to PathFinder's modular monolith. It is a proposed starting structure; Kiro must validate it against the selected Next.js version and installed Kiro conventions before creating files.

```
pathfinder/
|-- .kiro/
|   |-- steering/              # persistent product, tech and structure guidance
|   `-- specs/                 # feature requirements, design and tasks
|-- docs/
|   |-- product/               # approved product references
|   |-- architecture/          # diagrams and decision records
|   `-- data/                  # data dictionary and verification guidance
|-- public/
|-- src/
|   |-- app/
|   |   |-- (public)/opportunities/
|   |   |-- (auth)/sign-in/
|   |   |-- (auth)/sign-up/
|   |   |-- dashboard/
|   |   |-- admin/
|   |   `-- api/               # only where route handlers are needed
|   |-- modules/
|   |   |-- identity/
```

```
|   |   |-- profiles/
|   |   |-- opportunities/
|   |   |-- eligibility/
|   |   |-- matching/
|   |   |-- readiness/
|   |   |-- applications/
|   |   `-- verification/
|   |-- db/
|   |   |-- schema/
|   |   |-- migrations/
|   |   `-- index.ts
|   |-- lib/
|   |   |-- ai/                 # DeepSeek service boundary and schemas
|   |   |-- auth/
|   |   |-- validation/
|   |   `-- utils/
|   |-- components/             # shared UI, not domain logic
|   `-- tests/
|       |-- unit/
|       |-- integration/
|       `-- e2e/
|-- .env.example
|-- AGENTS.md                   # if supported by agreed workflow
|-- drizzle.config.ts
|-- next.config.ts
|-- package.json
|-- tsconfig.json
`-- README.md
```

Structural rules

- Keep domain logic inside feature modules, not page components.
- Eligibility must be testable without rendering UI or calling an LLM.
- AI service accepts typed inputs and returns validated structured outputs.
- Use one documented database schema/migration path.
- Verify Kiro's actual support for steering/spec/agent conventions before assuming exact paths or features.

9. Functional requirements

Preserve these identifiers exactly for traceability. Kiro must not renumber or silently merge them.

ID	Requirement
FR-01	Visitors must be able to browse published opportunities without creating an account.
FR-02	Visitors must be able to search and filter opportunities using supported structured attributes.
FR-03	The system must provide opportunity detail pages with source and application links.
FR-04	Users must be able to register, authenticate and manage their account.
FR-05	Users must be able to create and update a profile appropriate to undergraduate or recent-graduate status.
FR-06	Profiles must support relevant education, nationality, residence, skills, experience, career goals and NYSC status where applicable.
FR-07	The catalogue must support scholarships, SIWES/IT, professional internships, fellowships, graduate programmes, funded training and leadership opportunities.
FR-08	The system must distinguish opportunity destination from eligible applicant nationality and residence.
FR-09	The system must represent funding components independently and allow unknown/not-stated values.
FR-10	Opportunity records must store official source information, lifecycle status and last verification timestamp.
FR-11	Authorized administrators must be able to create, edit, review, publish, close and archive opportunities.
FR-12	Eligibility criteria must be stored in a structured form where feasible, including mandatory/optional status.
FR-13	The eligibility engine must compare profile data against known mandatory criteria.
FR-14	The system must distinguish eligible, likely eligible, needs review and not eligible outcomes.
FR-15	The system must show criterion-level explanations and distinguish unmet requirements from unknown data.
FR-16	The matching engine must calculate relevance separately from eligibility.

ID	Requirement
FR-17	The system must explain the main factors contributing to a recommendation.
FR-18	Users must be able to request AI-assisted explanations and readiness plans.
FR-19	AI outputs must be validated and must not invent opportunity requirements, deadlines or funding facts.
FR-20	Users must be able to save and unsave opportunities.
FR-21	Users must be able to record and update application progress.
FR-22	The system must link users to the official provider application channel.
FR-23	The system must prevent expired or unverified listings from being presented as confirmed open opportunities.
FR-24	The system must allow users to report an inaccurate or expired opportunity (implementation priority to be confirmed).

10. Technical requirements

ID	Requirement
TR-01	Implement the application in Next.js with TypeScript strict mode (proposed stack; confirm exact framework version).
TR-02	Use PostgreSQL as the primary relational data store.
TR-03	Use a typed ORM and version-controlled database migrations.
TR-04	Validate API inputs and structured AI outputs at runtime.
TR-05	Enforce authorization on the server for every protected operation; hiding UI controls is not authorization.
TR-06	Keep user profile, assessment and application data private to the owning user and authorized administrators.
TR-07	Implement eligibility checks in deterministic application logic wherever criteria are structured.
TR-08	Keep eligibility, relevance ranking and readiness analysis as separate domain operations.
TR-09	Route LLM calls through an internal service boundary and validate outputs against schemas; use DeepSeek as the sole provider through the Vercel AI SDK.
TR-10	Store source provenance and verification timestamps for published opportunity facts.
TR-11	Represent missing, unknown and ambiguous facts explicitly.
TR-12	Use database constraints and indexes for integrity and common catalogue queries.
TR-13	Keep secrets and provider API keys on the server; never expose them in client bundles.
TR-14	Use environment variables and provide an example environment file without real secrets.
TR-15	Maintain automated tests for eligibility rules, authorization boundaries and critical workflows.
TR-16	Use accessible, responsive UI patterns for mobile and desktop.
TR-17	Record relevant timestamps in a consistent timezone strategy and render deadlines in a clearly identified timezone.
TR-18	Do not introduce vector search, Redis queues or separate services without a documented need.

11. Non-functional expectations

Area	Expectation
Security	Secure authentication, server-side authorization, protected secrets, HTTPS and least-privilege access.
Privacy	Collect only profile data needed for functionality; provide account/data management and privacy policy before public launch.
Accuracy	Eligibility outcomes trace to criteria and profile facts. Unknown facts never become confirmed results.
AI safety	No fabricated eligibility, deadlines, funding, provider decisions or guarantees.
Freshness	Published opportunities display last-verified date and are not labelled open after confirmed closure.
Performance	Prior proposed target: ordinary catalogue search within 2 seconds under normal MVP load. Measure; do not claim as a guarantee.
Availability	Public catalogue and deterministic checks remain usable if DeepSeek is unavailable.
Usability	Core discovery/detail pages work on common mobile and desktop screen sizes.
Maintainability	Typed domain boundaries, modular code, documented setup and automated tests.
Scalability	Support more countries, opportunity types and cohorts without redesigning core entities.
Explainability	Show factors behind eligibility and relevance; avoid opaque certainty.

12. MVP roadmap

Milestone	Deliverable / exit condition
0. Foundation/specification	Approved requirements, architecture, data model, acceptance criteria and Kiro workflow.
1. Project foundation	Repository, Next.js/TypeScript setup, database connection, migrations, linting, tests and deployment skeleton.
2. Identity/profiles	Authentication, protected routes and undergraduate/recent-graduate profile onboarding.
3. Catalogue	Admin CRUD, structured opportunities, public detail pages, search and filters.
4. Verification	Source records, verification history, lifecycle states and publication controls.
5. Eligibility	Structured criteria, deterministic evaluation, criterion outcomes and tests.
6. Matching	Transparent relevance ranking and personalised results.
7. AI readiness	Validated explanations and readiness plans grounded in profile and source evidence using DeepSeek.
8. Saved/tracking	Bookmarks, application statuses, responsive polish, security review and MVP release.

MVP includes public catalogue, profile onboarding, curated opportunity database, search/filter, verification administration, eligibility checks, relevance ranking, AI readiness plans, saved opportunities and basic application tracking.

Deferred: large-scale crawling; CV parsing/upload; WhatsApp/email reminders; automated application submission; CV/cover-letter generation; AI interview coaching; employer dashboards; university integrations; multi-agent orchestration; advanced RAG/vector search; separate worker infrastructure unless justified.

13. Risks and validation points

Risk	Handling / why it matters
Opportunity freshness	Stale deadlines and eligibility damage trust. Start with manual curation; define review cadence from capacity.
Global eligibility complexity	Nationality, residence, visa, age, language and education rules vary. Model separately and use review states.
Ambiguous criteria	Preserve source evidence and require review when rules cannot be safely structured.
Matching weights	Arbitrary scores mislead. Use transparent factors and validate usefulness before tuning.
DeepSeek quality/cost/latency	Provider selected; exact model, limits, structured output, cost and latency require testing.
Authentication/hosting	Choose one auth provider and database host before implementation.
Funding representation	Funding is multi-component and sometimes incomplete; store each component and evidence state.
Deadline/timezone/cycles	Preserve timezone context and model recurring programme cycles where necessary.
Differentiation	Validate Nigerian undergraduate/early-career focus and trusted eligibility experience through user research.

14. Agent rules to extract into AGENTS.md or equivalent

The following is the source material for the repository's agent instructions. The agent file should be a concise operational copy, not a competing product specification.

- Do not guess: identify missing information, conflicting requirements and unsupported assumptions; ask before consequential decisions.
- Preserve scope: do not add, remove or reinterpret capabilities without approval.
- Eligibility is deterministic: DeepSeek never decides eligibility; relevance cannot override mandatory failure.
- Ground AI: use verified opportunity facts and relevant profile data; never invent requirements, dates, funding, links or outcomes.
- Unknown stays unknown: missing data is not a pass, fail or confirmed fact.
- Protect users: enforce ownership and admin authorization server-side; keep secrets server-side.
- Respect architecture: modular monolith; no microservices, worker queues, vector search or extra providers without documented need and approval.
- Review first: present database schema, migrations strategy, auth choice and consequential architecture decisions for approval.
- Small verified changes: bounded tasks, traceable requirements, relevant tests and evidence-based completion reports.
- Keep documentation aligned: update specs and decision records when an approved decision changes.

15. Instructions for Kiro's specification phase

- Read this full reference and the tutor-template-based project brief before generating project files.
- Preserve FR-01 through FR-24 and TR-01 through TR-18 exactly.
- Use the tutor's supplied template; do not invent its required format.
- Check installed Kiro capabilities and use supported conventions for steering, specs and agent instructions.
- Propose and explain the database schema from the conceptual entity inventory; stop for approval before migrations.
- Confirm exact framework/package versions, authentication provider, hosting/database provider, package manager and DeepSeek model/API compatibility.

• Do not begin feature implementation until requirements, architecture, data model and the first bounded task are reviewed and approved.

End of PathFinder Project Requirements & Technical Reference